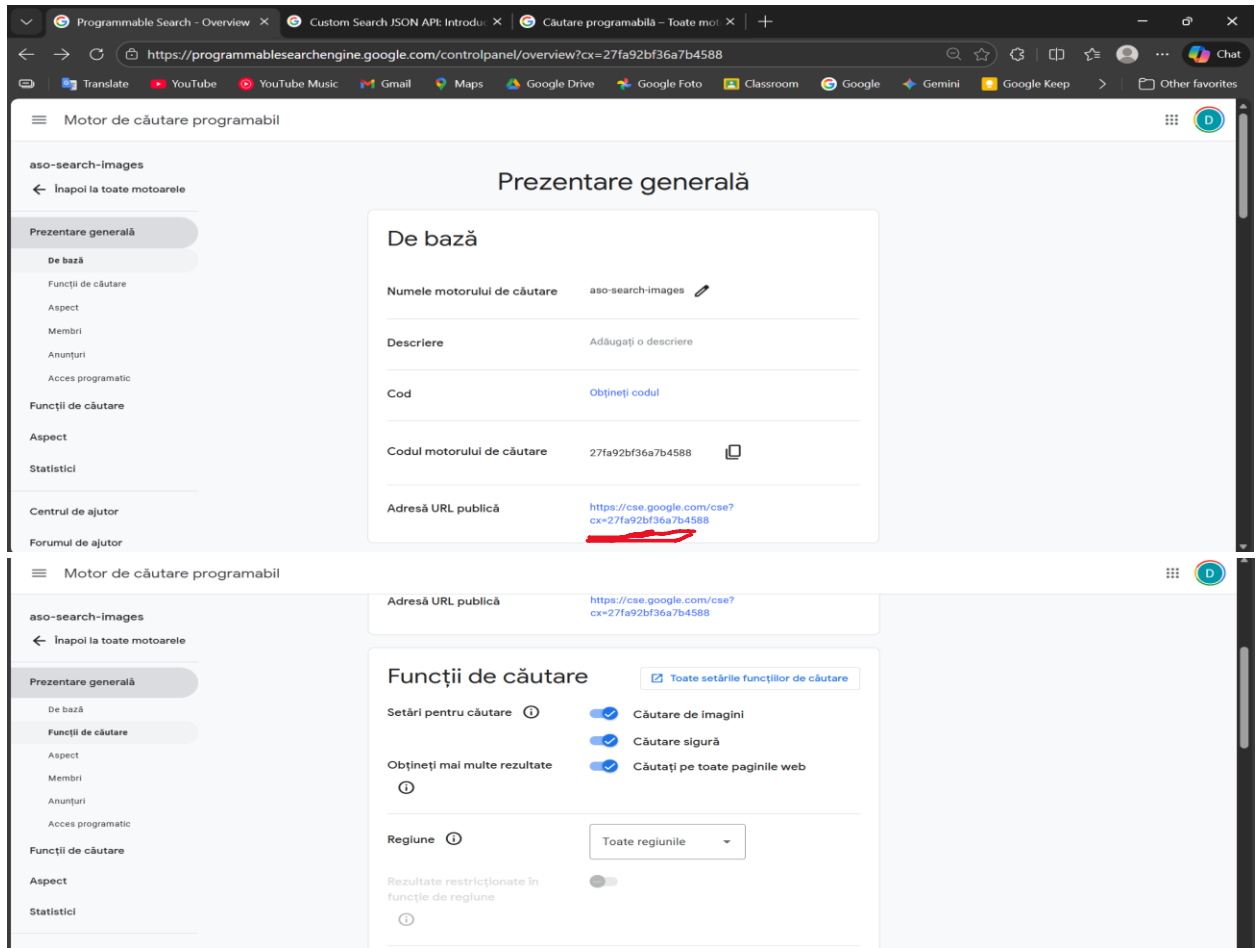
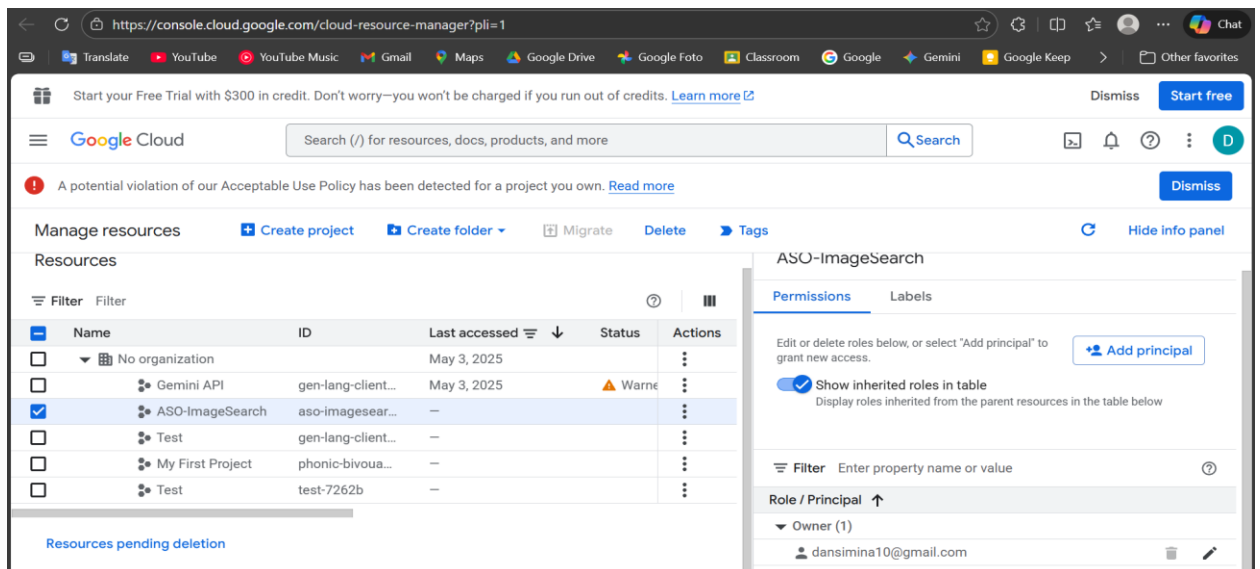


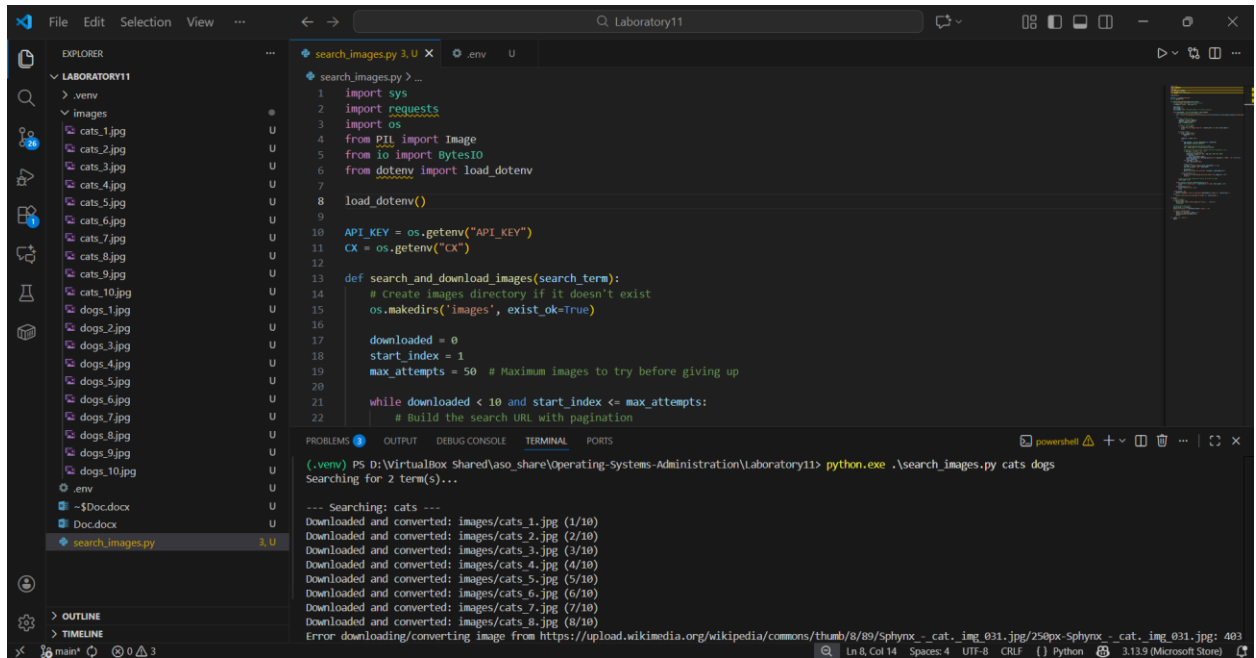
1. Creați la adresa un motor custom de căutare, numit "aso-search-images".



2. Accesați Google API Console și creați un proiect nou numit "aso-search-images" (crearea durează câteva zeci de secunde!).



3. Folosiți biblioteca Python "requests" pentru a face căutări de imagini.



```
search_images.py 3.U
1 import sys
2 import requests
3 import os
4 from PIL import Image
5 from io import BytesIO
6 from dotenv import load_dotenv
7
8 load_dotenv()
9
10 API_KEY = os.getenv("API_KEY")
11 CX = os.getenv("CX")
12
13 def search_and_download_images(search_term):
14     # Create images directory if it doesn't exist
15     os.makedirs('images', exist_ok=True)
16
17     downloaded = 0
18     start_index = 1
19     max_attempts = 50 # Maximum images to try before giving up
20
21     while downloaded < 10 and start_index <= max_attempts:
22         # Build the search URL with pagination
23         url = f'https://www.googleapis.com/customsearch/v1?q={search_term}&cx={CX}&key={API_KEY}&start-index={start_index}&max-results=10'
24         response = requests.get(url)
25         if response.status_code == 200:
26             results = response.json().get('items', [])
27             if results:
28                 for result in results:
29                     img_url = result['image']['url']
30                     img_response = requests.get(img_url)
31                     img = Image.open(BytesIO(img_response.content))
32                     # Convert to RGB if necessary (handles PNG with transparency, etc.)
33                     if img.mode in ('RGBA', 'LA', 'P'):
34                         # Create white background
35                         background = Image.new('RGB', img.size, (255, 255, 255))
36                         if img.mode == 'P':
37                             img = img.convert('RGBA')
38                         background.paste(img, mask=img.split()[-1] if img.mode in ('RGBA', 'LA') else None)
39                         img = background
40                     elif img.mode != 'RGB':
41                         img = img.convert('RGB')
42                     # Save as JPG
43                     filename = f'images/{search_term}_{downloaded + 1}.jpg'
44                     img.save(filename, 'JPEG', quality=95)
45                     downloaded += 1
46                     start_index += 10
47             else:
48                 start_index += 1
49         else:
50             start_index += 1
51
52 if __name__ == '__main__':
53     search_term = sys.argv[1]
54     search_and_download_images(search_term)
```

Terminal Output:

```
(.venv) PS D:\VirtualBox Shared\aso_share\Operating-Systems-Administration\Laboratory11> python.exe .\search_images.py cats dogs
Searching for 2 term(s)...
--- Searching: cats ---
Downloaded and converted: images/cats.1.jpg (1/10)
Downloaded and converted: images/cats.2.jpg (2/10)
Downloaded and converted: images/cats.3.jpg (3/10)
Downloaded and converted: images/cats.4.jpg (4/10)
Downloaded and converted: images/cats.5.jpg (5/10)
Downloaded and converted: images/cats.6.jpg (6/10)
Downloaded and converted: images/cats.7.jpg (7/10)
Downloaded and converted: images/cats.8.jpg (8/10)
Error downloading/convert image from https://upload.wikimedia.org/wikipedia/commons/thumb/8/89/Sphynx_-_cat_-_img_031.jpg/258px-Sphynx_-_cat_-_img_031.jpg: 403
```

4. Daca nu sunt imagini în format jpg, le puteti salva în format jpg.

```
# Open image with PIL and convert to RGB
img = Image.open(BytesIO(img_response.content))

# Convert to RGB if necessary (handles PNG with transparency, etc.)
if img.mode in ('RGBA', 'LA', 'P'):
    # Create white background
    background = Image.new('RGB', img.size, (255, 255, 255))
    if img.mode == 'P':
        img = img.convert('RGBA')
    background.paste(img, mask=img.split()[-1] if img.mode in ('RGBA', 'LA') else None)
    img = background
elif img.mode != 'RGB':
    img = img.convert('RGB')

# Save as JPG
filename = f"images/{search_term}_{downloaded + 1}.jpg"
img.save(filename, 'JPEG', quality=95)
```

COD:

```
import sys
import requests
import os
from PIL import Image
from io import BytesIO
from dotenv import load_dotenv

load_dotenv()

API_KEY = os.getenv("API_KEY")
CX = os.getenv("CX")

def search_and_download_images(search_term):
    # Create images directory if it doesn't exist
    os.makedirs('images', exist_ok=True)

    downloaded = 0
    start_index = 1
    max_attempts = 50 # Maximum images to try before giving up

    while downloaded < 10 and start_index <= max_attempts:
        # Build the search URL with pagination
        url = f"https://www.googleapis.com/customsearch/v1?key={API_KEY}&cx={CX}&q={search_term}&searchType=image&imgSize=large&start={start_index}"

        try:
            # Make request to API
            response = requests.get(url)
            response.raise_for_status()
            data = response.json()

            # Check if results exist
            if 'items' not in data:
                print(f"No more images found for '{search_term}' at index {start_index}")
                break

            # Process results
            for item in data['items']:
                if downloaded >= 10:
                    break

                image_url = item['link']

                try:
                    img_response = requests.get(image_url, timeout=10)
                    img_response.raise_for_status()

                    # Open image with PIL and convert to RGB
                    img = Image.open(BytesIO(img_response.content))

                    # Convert to RGB if necessary (handles PNG with transparency, etc.)
                    if img.mode in ('RGBA', 'LA', 'P'):
                        # Create white background
                        background = Image.new('RGB', img.size, (255, 255, 255))
                        if img.mode == 'P':
                            img = img.convert('RGBA')
                        background.paste(img, mask=img.split()[-1] if img.mode in ('RGBA', 'LA') else None)
                        img = background
                    elif img.mode != 'RGB':
                        img = img.convert('RGB')

                    # Save as JPG
                    filename = f"images/{search_term}_{downloaded + 1}.jpg"
                    img.save(filename, 'JPEG', quality=95)

                    downloaded += 1
                    print(f"Downloaded and converted: {filename} ({downloaded}/10)")

                except Exception as e:
                    print(f"Error downloading/converting image from {image_url}: {e}")
                    continue

            # Move to next page (Google API returns 10 results per page)
            start_index += 10

        except requests.exceptions.RequestException as e:
            print(f"Error searching for '{search_term}' at index {start_index}: {e}")
            break
        except Exception as e:
            print(f"General error: {e}")
            break

    if downloaded < 10:
        print(f"\nWARNING: Could only download {downloaded}/10 images for '{search_term}'")
    else:
        print(f"\nSuccessfully downloaded 10 images for '{search_term}'")

def main():
    # Check arguments
    if len(sys.argv) < 2:
```

```
        print("Usage: python search_images.py <term_1> ... <term_n>")
        sys.exit(1)

    # Process each search term
    search_terms = sys.argv[1:]
    print(f"Searching for {len(search_terms)} term(s)...\n")

    for term in search_terms:
        print(f"--- Searching: {term} ---")
        search_and_download_images(term)
        print()

if __name__ == "__main__":
    main()
```